

Quiz 3

601.471/671 NLP: Self-Supervised Models

Johns Hopkins University Fall 2026

Name: _____

Student version

Time: 15 minutes. Total: 100 points + 15 possible extra-credit points.

Complete Q1–Q2 for 100 points. Q3 is optional: leaving it blank does not reduce your score; earned bonus points are added beyond 100. The 15-minute limit includes any bonus work. For every item, select exactly one answer; no explanation is needed. No calculator is needed.

Q1. Training and Tokenization

40 points

- (a) (10 points) A hidden sigmoid unit has a very large positive preactivation. Why can learning through this unit become slow?
- ☐ A. Its local derivative is close to zero.
 - ☐ B. Its local derivative becomes arbitrarily large.
 - ☐ C. Its output must be exactly zero.
 - ☐ D. Its weights are automatically frozen.
- (b) (10 points) A byte-level BPE tokenizer includes all 256 byte values as base tokens. A valid UTF-8 word was absent from tokenizer training. Which statement is correct?
- ☐ A. It must become an unknown token.
 - ☐ B. It must become exactly one token.
 - ☐ C. It can be represented using byte tokens and any applicable learned merges.
 - ☐ D. It requires learning new merges during inference.
- (c) (10 points) A character-level BPE tokenizer has base tokens $\{x, y, z, w\}$ and has learned exactly these rules, in order:

$$y + z \rightarrow yz, \quad x + y \rightarrow xy, \quad z + w \rightarrow zw.$$

How does it tokenize the single chunk $xyzw$? Vertical bars below separate tokens.

- ☐ A. $[xy \mid zw]$
 - ☐ B. $[x \mid yz \mid w]$
 - ☐ C. $[xyzw]$
 - ☐ D. It must learn new rules first.
- (d) (10 points) A pipeline first lowercases text, then applies a reversible byte-level tokenizer. It stores no other information. Can decoding always recover the original capitalization?
- ☐ A. Yes, because all 256 bytes are supported.
 - ☐ B. Yes, if the vocabulary is large enough.
 - ☐ C. No, because lowercasing can make different inputs identical.
 - ☐ D. No, because BPE merges always destroy information.

Q2. Gradient Accumulation

60 points

We want one update using the mean loss over 32 examples, but only 8 examples fit in memory. Split them into four microbatches of 8. Each `mean_loss` below is the mean over its microbatch.

Assume per-example losses are independent of other examples, with no dropout, batch-dependent operations, or extra loss terms.

```
optimizer.zero_grad()

for x, y in four_microbatches:
    mean_loss = criterion(model(x), y) # mean over 8 examples
    (mean_loss / ____).backward()

optimizer.step()
```

For each item, select exactly one answer; no explanation is needed.

- (a) (20 points) Which value fills the blank so the accumulated gradient equals the gradient of the mean loss over all 32 examples?
- ☐ A. 1
 - ☐ B. 4
 - ☐ C. 8
 - ☐ D. 32
- (b) (20 points) At the same starting parameter value $w = 5$, the four unscaled microbatch mean losses have derivatives 2, 6, -2 , and 2 with respect to w . With the correct scaling and plain SGD with learning rate 0.1 (no momentum or weight decay), which pair gives the accumulated derivative g and the updated value w' ?
- ☐ A. $g = 8, w' = 4.2$
 - ☐ B. $g = 2, w' = 5.2$
 - ☐ C. $g = 0.5, w' = 4.95$
 - ☐ D. $g = 2, w' = 4.8$
- (c) (20 points) Which statement correctly explains both the lower peak activation memory and the effect of adding `optimizer.zero_grad()` inside the loop immediately before each `backward()`?
- ☐ A. Saved activations can be freed after each microbatch's backward pass; clearing gradients each time discards earlier microbatch contributions.
 - ☐ B. Saved activations can be freed after each microbatch's backward pass; clearing gradients each time preserves all microbatch contributions.
 - ☐ C. Microbatching reduces the number of model parameters; clearing gradients each time discards earlier microbatch contributions.
 - ☐ D. Microbatching reduces the number of model parameters; clearing gradients each time preserves all microbatch contributions.

Q3. Optional Bonus: Training Code

15 bonus points

For each item, select exactly one answer; no explanation is needed.

- (a) (5 points) For predictions of shape $(4, 1)$ and targets of shape $(4,)$, what is the shape of predictions - targets under PyTorch elementwise broadcasting?
- ☐ A. $(4,)$
 - ☐ B. $(4, 1)$
 - ☐ C. $(4, 4)$
 - ☐ D. The operation must raise a shape error.
- (b) (5 points) A model contains dropout. Which setup disables dropout and prevents recording a backward graph during validation?
- ☐ A. `model.eval()` alone
 - ☐ B. `model.eval()` together with `torch.no_grad()`
 - ☐ C. `torch.no_grad()` alone
 - ☐ D. `optimizer.zero_grad()` alone
- (c) (5 points) For an input tensor of shape $(2, 3, 4)$, `LayerNorm(4)` computes a separate mean over which values?
- ☐ A. All 24 values together
 - ☐ B. The two examples, for each position and feature
 - ☐ C. The three positions, for each example and feature
 - ☐ D. The four features, for each example and position